TCS Interview: DB + SQL

Topic 1: SQL Basics

What is SQL?

👉 **SQL (Structured Query Language)** is used to:

- store
- retrieve
- manipulate data in a database
  "SQL is a language used to interact with relational databases to perform operations like inserting, updating, deleting, and retrieving data."

Types of SQL operations (VERY IMPORTANT)

1. DDL (Data Definition Language)

👉 Deals with **structure**

- CREATE
- ALTER
- DROP
- TRUNCATE

2. DML (Data Manipulation Language)

👉 Deals with **data**

- INSERT
- UPDATE
- DELETE

3. DQL (Data Query Language)

👉 Fetch data

- SELECT

4. DCL (rarely asked)

- GRANT
- REVOKE

Example table

```
Students(id, name, marks)
```

Common queries (you MUST know)

Insert

```
INSERT INTO Students VALUES (1, 'John', 90);
```

Select

```
SELECT * FROM Students;
```

Update

```
UPDATE Students SET marks = 95 WHERE id = 1;
```

Delete

```
DELETE FROM Students WHERE id = 1;
```

Difference: DELETE vs TRUNCATE vs DROP

Clean answer:

DELETE

- removes specific rows
- uses WHERE
- can rollback

TRUNCATE

- removes all rows
- faster
- cannot rollback
- structure stays

DROP

- deletes entire table
- structure + data gone

Interview trick

If you say:

“TRUNCATE is faster than DELETE”

👉 You sound good already

What is a relational database?

“A relational database stores data in tables with rows and columns and maintains relationships between them.”

📌 What is a JOIN?

👉 Used to **combine data from multiple tables based on a condition**

💡 Example Tables

Students

```
id | name
1  | A
2  | B
3  | C
```

Marks

```
id | marks
1  | 90
2  | 80
4  | 70
```

Types of JOINS (you MUST know 4)

1. INNER JOIN

👉 Only matching rows

```
SELECT s.name, m.marks
FROM Students s
INNER JOIN Marks m
ON s.id = m.id;
```

👉 Output:

- A → 90
- B → 80

2. LEFT JOIN

👉 All from left + matching from right

```
SELECT s.name, m.marks
FROM Students s
LEFT JOIN Marks m
ON s.id = m.id;
```

👉 Output:

- A → 90
- B → 80
- C → NULL

3. RIGHT JOIN

👉 All from right + matching from left

```
SELECT s.name, m.marks
FROM Students s
RIGHT JOIN Marks m
ON s.id = m.id;
```

👉 Output:

- A → 90
- B → 80
- NULL → 70

4. FULL JOIN (less common in MySQL)

👉 All rows from both

INTERVIEW TIP

👉 If confused:

- INNER = common
- LEFT = everything from left
- RIGHT = everything from right

NOW — MEDIUM LEVEL QUERIES (THIS IS WHAT YOU WANTED)

1. Find students who DO NOT have marks

👉 Classic LEFT JOIN trap

```
SELECT s.name
FROM Students s
LEFT JOIN Marks m
```

```
ON s.id = m.id
WHERE m.id IS NULL;
```

2. Find duplicate values

```
SELECT name, COUNT(*)
FROM Students
GROUP BY name
HAVING COUNT(*) > 1;
```

3. Second highest salary

```
SELECT MAX(salary)
FROM Employees
WHERE salary < (SELECT MAX(salary) FROM Employees);
```

4. Third highest salary (medium level)

```
SELECT DISTINCT salary
FROM Employees
ORDER BY salary DESC
LIMIT 1 OFFSET 2;
```

5. Count number of students per department

```
SELECT dept_id, COUNT(*) AS total
FROM Students
GROUP BY dept_id;
```

6. Find employees with salary > average

```
SELECT name, salary
FROM Employees
WHERE salary > (SELECT AVG(salary) FROM Employees);
```

7. Self Join (IMPORTANT)

👉 Find employees with same salary

```
SELECT e1.name, e2.name
FROM Employees e1
JOIN Employees e2
ON e1.salary = e2.salary
AND e1.id != e2.id;
```

8. Find max salary in each department

```
SELECT dept_id, MAX(salary)
FROM Employees
GROUP BY dept_id;
```

IMPORTANT CONCEPTS YOU JUST USED

👉 GROUP BY
👉 HAVING
👉 Subqueries
👉 JOIN logic

HIGH-PROBABILITY FOLLOW UPS

Difference: WHERE vs HAVING

- WHERE → filters rows before grouping
- HAVING → filters after grouping

What is a subquery?

“A query inside another query used to retrieve intermediate results.”

What is self join?

“Joining a table with itself.”

1. Normalization (VERY IMPORTANT but EASY marks)

👉 **Process of organizing data to reduce redundancy and improve consistency**

“Normalization is the process of structuring a database to eliminate redundancy and maintain data integrity.”

Why we need it

Without normalization:

- Duplicate data ❌
- Inconsistency ❌
- Update issues ❌

Types (only remember these 3)

1NF (First Normal Form)

👉 Rules:

- No multiple values in a single column

- Atomic values only

❌ Wrong:

```
id | subjects
1  | Math, Science
```

✅ Correct:

```
id | subject
1  | Math
1  | Science
```

2NF (Second Normal Form)

👉 Rule:

- Must be in 1NF
- No partial dependency

👉 Means:
All columns must depend on **full primary key**

💡 Simple idea:

- If table has composite key → don't depend on only part of it

3NF (Third Normal Form)

👉 Rule:

- Must be in 2NF
- No transitive dependency

👉 Means:
Non-key columns should NOT depend on other non-key columns

💡 Example:

❌ Wrong:

```
id | dept_id | dept_name
```

👉 dept_name depends on dept_id (not id)

✅ Fix: Separate table for departments

Shortcut to remember

Form	Rule
1NF	no multi-values

2NF no partial dependency
3NF no indirect dependency

Interview tip

They won't go deep.

If asked:
👉 Explain 1NF, 2NF, 3NF in **simple language**

2. Indexing (short but powerful)

What is Index?

👉 **Index is used to speed up data retrieval**

"An index is a data structure that improves the speed of data retrieval operations on a table."

Example

Without index:
👉 DB scans entire table (slow)

With index:
👉 DB jumps directly (fast)

Trade-off (IMPORTANT)

👉 Index improves: SELECT speed ✅

👉 But: INSERT / UPDATE slower ❌

Real-life analogy

👉 Like index in a book
Instead of reading whole book → jump to page

Follow-up questions

Where do we use index?

- frequently searched columns
- primary keys

Disadvantages?

- extra storage

- slower writes

ACID Properties (DB — important)

What is ACID?

👉 Properties that ensure **reliable transactions in a database**

A – Atomicity

👉 “All or nothing”

- If one part fails → whole transaction fails
- Example: money transfer

C – Consistency

👉 Database stays valid

- Rules/constraints maintained
- No invalid data

I – Isolation

👉 Transactions don’t interfere

- Multiple users → no conflict

D – Durability

👉 Once committed → permanent

- Even if system crashes

Interview answer

“ACID properties ensure reliable and consistent database transactions by enforcing atomicity, consistency, isolation, and durability.”